

CC3092 – Deep Learning

Laboratorio 7

Instrucciones

- Esta es una actividad en grupos de no más de 3 integrantes.
 - Recuerden **unirse al grupo de canvas**
- No se permitirá ni se aceptará cualquier indicio de copia. De presentarse, se procederá según el reglamento correspondiente.
- Tendrán hasta el día indicado en Canvas.
 - No se confíen, aprovechen el tiempo en clase para entender todos los ejercicios y avanzar lo más posible.
- **NOTA:** Limiten el uso de IA generativa. Intenten primero buscar en fuentes de internet y si en verdad necesitan usarla, asegúrense de **colocar el prompt** que utilizan para cada task donde corresponda, así como una explicación de por qué ese prompt funcionó.

Este laboratorio implementa desde cero un sistema de forecasting sobre el dataset ETTh1 (Electricity Transformer Temperature), que registra la temperatura del aceite de transformadores de potencia eléctrica en China a resolución horaria durante dos años. La variable objetivo es OT (Oil Temperature), cuya predicción determina si un transformador está en riesgo de falla térmica.

Usted implementará dos arquitecturas con tensores PyTorch: un LSTM many-to-one y un Transformer encoder con self-attention temporal. Ambas se evaluarán sobre dos horizontes de predicción: $h = 24$ horas y $h = 48$ horas.

NOTA: Todo el código de las secciones marcadas debe implementarse con tensores PyTorch puros. No se permite usar `nn.LSTM`, `nn.MultiheadAttention` ni ninguna capa de alto nivel que implemente la arquitectura por usted. Puede usar `nn.Linear`, `nn.Parameter`, `torch.optim.Adam` y `loss.backward()` normalmente.

Task 1 (Entrega Parcial)

Task 1.1

Descargue ETTh1 directamente desde GitHub y construya el pipeline de preprocesamiento.

```
import urllib.request
import numpy as np
import torch

URL = ("https://raw.githubusercontent.com/zhouhaoyi/"
      "ETDataset/main/ETT-small/ETTh1.csv")
urllib.request.urlretrieve(URL, "ETTh1.csv")

data = np.genfromtxt("ETTh1.csv", delimiter=",", skip_header=1)
OT = data[:, -1].astype(np.float32) # Oil Temperature: columna objetivo
ALL = data[:, 1:].astype(np.float32) # 7 variables (6 de carga + OT)
```

- a. Normalice OT restando su media y dividiendo por su desviación estándar. Guarde μ y σ para desnormalizar las predicciones al evaluar.
- b. Implemente el split temporal con los siguientes porcentajes sobre OT normalizada: 60% entrenamiento, 20% validación, 20% prueba. El split debe ser un bloque temporal contiguo para cada conjunto, sin mezcla aleatoria.

- c. Implemente la funcion `make_windows(series, L, H)` que recibe una serie 1D, la longitud de ventana L y el horizonte H , y retorna tensores X de forma (N, L) e y de forma $(N,)$ donde cada par corresponde a:

$$X^{(t)} = (x_{t-L+1}, \dots, x_t), \quad y^{(t)} = x_{t+H}$$

Aplique `make_windows` sobre cada split con $L = 96$ y $H \in \{24, 48\}$.

NOTA: El valor $L = 96$ corresponde a cuatro dias de historia horaria. En el Task 1.2 usted justificara si ese valor es apropiado usando la ACF.

Task 1.2

- a. Implemente `acf_manual(series, max_lag)` que calcule la funcion de autocorrelacion de la serie sobre los lags $0, 1, \dots, \text{max_lag}$ usando la formula:

$$\rho(k) = \frac{\frac{1}{N-k} \sum_{t=k}^{N-1} (x_t - \bar{x})(x_{t-k} - \bar{x})}{\frac{1}{N} \sum_{t=0}^{N-1} (x_t - \bar{x})^2}$$

Calcule la ACF sobre el conjunto de entrenamiento con `max_lag=96` y grafiquela con lineas de confianza al 95% en $\pm 1.96/\sqrt{N}$.

- Identifique el lag con el segundo pico mas alto de la ACF (despues de lag 0). Interprete ese valor en terminos de la periodicidad de la temperatura del aceite.
- Con base en el correlograma obtenido, justifique si $L = 96$ es una eleccion apropiada para la longitud de ventana o si deberia ser mayor o menor. Su respuesta debe hacer referencia explicita a los valores de la ACF.

Verificacion Task 1:

```
# Ejecute esta celda para verificar su implementacion
_ok = True

# V1: split temporal correcto
assert len(OT_train) + len(OT_val) + len(OT_test) == len(OT_norm), \
    "Los splits no cubren toda la serie"
assert OT_train[-1] != OT_val[0], \
    "El ultimo valor de train no debe ser el primero de val"

# V2: ventanas correctas
assert X_tr_24.shape[1] == 96, f"Ventana incorrecta: {X_tr_24.shape[1]}"
assert y_tr_24.shape[0] == X_tr_24.shape[0], "X e y tienen distinto numero de ejemplos"

# V3: ACF en lag 24 (estacionalidad diaria esperada)
acf_lag24 = acf_manual(OT_train.numpy(), 48)[24]
assert 0.875 < acf_lag24 < 0.975, \
    f"ACF lag 24 fuera de rango: {acf_lag24:.4f} (esperado en [0.875, 0.975])"

# V4: baseline naive
naive_mae = torch.abs(y_vl_24 - X_vl_24[:, -1]).mean().item()
assert 0.15 < naive_mae < 0.25, \
    f"Naive MAE fuera de rango: {naive_mae:.4f} (esperado en [0.15, 0.25])"

print(f"Task 1: CORRECTO (ACF lag24={acf_lag24:.4f}, naive_mae={naive_mae:.4f})")
```



Task 2 (Entrega Parcial)

Task 2.1

Implemente la clase LSTMForecaster con tensores PyTorch. La celda LSTM debe procesarse manualmente paso a paso: no use nn.LSTM.

```
import torch.nn as nn
import torch.nn.functional as F

class LSTMForecaster(nn.Module):
    def __init__(self, d_in, d_h):
        super().__init__()
        self.d_h = d_h
        # SU CODIGO: declare los pesos de las 4 compuertas
        # Wx: (4*d_h, d_in), Wh: (4*d_h, d_h), b: (4*d_h,)
        # W_out: (1, d_h), b_out: (1,)
        # Use nn.Parameter para que sean aprendibles

    def forward(self, x):
        """
        Parametros
        -----
        x : Tensor (B, L): batch de secuencias univariadas

        Retorna
        -----
        pred : Tensor (B,): prediccion escalar por secuencia

        Pasos:
        1. Reshape x a (B, L, 1) para procesarlo paso a paso
        2. Inicializar h y c en ceros: forma (B, d_h)
        3. Para cada paso t en range(L):
            a. Calcular gates = x[:,t,:] @ Wx.T + h @ Wh.T + b
            b. Separar en i, f, g, o de dimension d_h cada una
            c. Aplicar sigmoid a i, f, o y tanh a g
            d. Actualizar c = f*c + i*g
            e. Actualizar h = o*torch.tanh(c)
        4. Proyectar h_T con W_out: pred = h @ W_out.T + b_out
        5. Retornar pred.squeeze(-1)
        """
        # SU CODIGO AQUI
        pass
```

Task 2.2

- Instancie LSTMForecaster(d_in=1, d_h=32) y entrene con torch.optim.Adam durante 20 epocas con batch_size=64. Use la función de perdida que considere apropiada para este problema y justifique su eleccion en las preguntas de analisis.
- Entrene dos modelos: uno para $H = 24$ y otro para $H = 48$ sobre el mismo conjunto de entrenamiento.
- Para cada modelo, calcule sobre el conjunto de validacion: MAE, RMSE y el ratio $\text{MAE_modelo} / \text{MAE_naive}$. Un ratio menor que 1 indica que el modelo supera al baseline.
- Grafique las curvas de perdida de entrenamiento para ambos horizontes en la misma figura.





Verificacion Task 2:

```
# Verificar shapes del forward
_model_test = LSTMForecaster(d_in=1, d_h=32)
_x_test = torch.randn(8, 96)
_pred_test = _model_test(_x_test)
assert _pred_test.shape == (8,), \
    f"Shape incorrecto: {_pred_test.shape} (esperado (8,))"

# Verificar que el modelo supera al naive en H=24
mae_lstm_24 = torch.abs(y_vl_24 - pred_vl_24).mean().item()
assert mae_lstm_24 < naive_mae * 1.1, \
    f"LSTM H=24 no supera al naive: {mae_lstm_24:.4f} vs {naive_mae:.4f}"

print(f"Task 2: CORRECTO")
print(f"  LSTM H=24: MAE={mae_lstm_24:.4f}, ratio={mae_lstm_24/naive_mae:.3f}")
```

Task 3 (Entrega Final)

Task 3.1

La entrada de cada paso temporal es un escalar $x_t \in \mathbb{R}$. Para que el Transformer pueda procesarla debe proyectarse a la dimensión del modelo d_{model} .

- Implemente `make_pe(max_len, d_model)` que calcule el positional encoding de Vaswani et al. (2017):

$$PE(t, 2i) = \sin\left(\frac{t}{10000^{2i/d_{model}}}\right), \quad PE(t, 2i + 1) = \cos\left(\frac{t}{10000^{2i/d_{model}}}\right)$$

Retorne un tensor de forma (L, d_{model}) no aprendible.

- La clase `TransformerForecaster` debe proyectar cada valor escalar x_t a $\mathbb{R}^{d_{model}}$ con una capa lineal `nn.Linear(1, d_model)` antes de sumar el positional encoding.

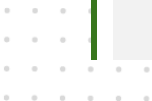
Task 3.2

Implemente el mecanismo de multi-head self-attention dentro de `TransformerForecaster`. No use `nn.MultiheadAttention`.

```
class TransformerForecaster(nn.Module):
    def __init__(self, d_model, n_heads, d_ff, L):
        super().__init__()
        self.d_model = d_model
        self.n_heads = n_heads
        self.d_k = d_model // n_heads
        self.L = L

        # SU CODIGO: declare los parametros
        # input_proj: nn.Linear(1, d_model)
        # WQ, WK, WV, WO: nn.Parameter de forma (d_model, d_model)
        # W1, b1: proyeccion FFN a d_ff
        # W2, b2: proyeccion FFN de vuelta a d_model
        # gamma1, beta1, gamma2, beta2: LayerNorm aprendible
        # W_out: nn.Linear(d_model, 1)

    def layer_norm(self, x, gamma, beta, eps=1e-5):
        # SU CODIGO: LayerNorm sobre la ultima dimension
        pass
```



```

def multi_head_attention(self, x):
    """
    Parametros
    -----
    x : Tensor (B, L, d_model)

    Retorna
    -----
    out      : Tensor (B, L, d_model)
    attn_w   : Tensor (B, n_heads, L, L): pesos de atencion

    Pasos:
    1. Proyectar con WQ, WK, WV: forma (B, L, d_model)
    2. Reshape a (B, L, n_heads, d_k) y transponer a (B, n_heads, L, d_k)
    3. scores = Q @ K^T / sqrt(d_k): forma (B, n_heads, L, L)
    4. attn_w = softmax(scores, dim=-1)
    5. out = attn_w @ V: forma (B, n_heads, L, d_k)
    6. Reshape a (B, L, d_model) y proyectar con W0
    """
    # SU CODIGO AQUI
    pass

def feed_forward(self, x):
    # SU CODIGO: FFN(x) = ReLU(x @ W1 + b1) @ W2 + b2
    pass

def forward(self, x, return_attn=False):
    """
    Parametros
    -----
    x           : Tensor (B, L)
    return_attn : bool: si True, retorna tambien los pesos de atencion

    Retorna
    -----
    pred      : Tensor (B,)
    attn_w    : Tensor (B, n_heads, L, L): solo si return_attn=True

    Pasos:
    1. x = input_proj(x.unsqueeze(-1)) + PE[:,L] forma (B, L, d_model)
    2. att_out, attn_w = multi_head_attention(x)
    3. x = layer_norm(x + att_out, gamma1, beta1) Add & Norm
    4. ff_out = feed_forward(x)
    5. x = layer_norm(x + ff_out, gamma2, beta2) Add & Norm
    6. pred = W_out(x[:, 0, :]).squeeze(-1) usar posicion 0 como CLS
    """
    # SU CODIGO AQUI
    pass

```

Task 3.3

- Instancie TransformerForecaster(d_model=32, n_heads=2, d_ff=64, L=96) y entrene con las mismas condiciones que el LSTM (20 epocas, Adam, batch_size=64). Entrene un modelo para $H = 24$ y otro para $H = 48$.
- Calcule MAE, RMSE y ratio vs naive para ambos horizontes.

- Extraiga los mapas de atencion para 5 ejemplos del conjunto de validacion usando `return_attn=True`. Grafique el mapa de atencion promedio (promedio sobre los 5 ejemplos y sobre las 2 cabezas) como un heatmap de forma (L, L) .
- Identifique cuales posiciones de la secuencia reciben mayor atencion desde la posicion 0 (el token CLS). Expresese esas posiciones como lags respecto al final de la ventana (lag 1 = el valor mas reciente, lag 96 = el valor mas antiguo).

Verificacion Task 3:

```
# Verificar shapes
_trf_test = TransformerForecaster(d_model=32, n_heads=2, d_ff=64, L=96)
_x_test = torch.randn(4, 96)
_pred_test, _attn_test = _trf_test(_x_test, return_attn=True)
assert _pred_test.shape == (4, ), \
    f"pred shape incorrecto: {_pred_test.shape}"
assert _attn_test.shape == (4, 2, 96, 96), \
    f"attn shape incorrecto: {_attn_test.shape}"

# Verificar que la atencion suma 1 por fila
row_sums = _attn_test[0, 0].sum(-1)
assert torch.allclose(row_sums, torch.ones(96), atol=1e-4), \
    "Los pesos de atencion no suman 1 por fila"

print(f"Task 3: CORRECTO")
print(f"  attn_w shape: {_attn_test.shape}")
print(f"  row sums OK: {row_sums.min().item():.4f} a {row_sums.max().item():.4f}")
```

Task 4 (Entrega Final)

Task 4.1

La ACF que calculó en el Task 1.2 muestra el valor $\rho(24)$ correspondiente a la estacionalidad diaria. En el mapa de atencion que extrajo en el Task 3.3, identifique los lags con mayor peso de atencion desde la posicion CLS.

- ¿Los lags con mayor atencion coinciden con los lags donde la ACF tiene sus picos mas altos? Muestre los valores numericos de ambos (ACF por lag y atencion por lag) para justificar su respuesta. Si no coinciden, proponga una hipotesis sobre por que el Transformer aprendio un patron de atencion distinto al que la ACF sugiere.
- La ACF es una medida lineal de dependencia: captura correlaciones lineales entre x_t y x_{t-k} . El mecanismo de atencion es no lineal porque los scores $e_{ts} = \mathbf{q}_t^\top \mathbf{k}_s / \sqrt{d_k}$ dependen de proyecciones aprendidas de los valores. Explique que tipo de dependencia entre pasos temporales podria capturar el Transformer que la ACF no puede detectar, y si sus mapas de atencion muestran alguna evidencia de ese tipo de dependencia.

Task 4.2

Compare los resultados de LSTM y Transformer en la siguiente tabla y luego responda las preguntas:

Modelo	Horizonte	MAE	RMSE	Ratio vs naive
LSTM	24h			
LSTM	48h			
Transformer	24h			

Transformer	48h			
Naive	24h			1.000
Naive	48h			1.000

- Al aumentar el horizonte de 24 a 48 horas, el error de ambos modelos aumenta. Explique matematicamente por que ese aumento es esperado en el caso del LSTM usando la formula de acumulacion de error en prediccion iterativa. En el caso del direct multi-output que usted implemento, el error no se acumula iterativamente, entonces por que tambien aumenta con el horizonte?
- Si el Transformer supera al LSTM en horizonte $H = 48$ pero no en $H = 24$, proponga una explicacion basada en la diferencia arquitectonica entre ambos modelos respecto al flujo del gradiente durante el entrenamiento. Use la formula del gradiente en el LSTM y en el Transformer para justificar su respuesta.

Entregas en Canvas

- Documento PDF con las respuestas a cada task
- En la entrega parcial se espera que entreguen lo **señalado**, En la entrega final deben entregar **TODOS LOS TASK**
- Archivo .ipynb, o link a repositorio de GitHub (**No se acepta entregas en otros medios**)
 - El código debe estar comentado explicando la relación con las fórmulas de las diapositivas

Evaluación

- [1.00 pt] Task 1
- [1.20 pt] Task 2
- [1.20 pt] Task 3
- [0.60 pt] Task 4

Total: 4.0 pts